

Project Report

MailSense: Actionable Email Detection Using NLP

Binary Classification of E-mail Text into Actionable and Non-Actionable

Submitted by

Md Ariful Alam Mahim

Roll: 2107023

Md Jubair Husain

Roll: 2107029

September 2026

Abstract

E-mail users routinely receive messages in which only some content requires them to act, reply or attend to something. This project, **MailSense**, addresses the automatic detection of such content as a binary text-classification task: each e-mail text is labelled either **Actionable (Yes)** or **Non-Actionable (No)**. We use the publicly available Parakweet Labs Email Intent Dataset (file Ask0729-fixed.txt, 3,657 labelled sentence-level examples) and compare three approaches representing three generations of NLP methods: (A) TF-IDF features with Logistic Regression, (B) a bidirectional LSTM initialised with pretrained Word2Vec embeddings, and (C) a fine-tuned pretrained BERT model (bert-base-uncased). All three models use the same cleaned data, the same stratified 70/15/15 train/validation/test split, the same random seed and the same evaluation code. On the held-out test set of 548 examples, BERT achieved the best results on every reported metric (accuracy 84.85%, precision 82.05%, recall 86.82%, F1 84.37%, ROC-AUC 93.16%) and produced the fewest missed actionable examples (34 false negatives). TF-IDF + Logistic Regression achieved 77.74% accuracy and 76.08% F1, while Word2Vec + BiLSTM achieved 74.82% accuracy and 74.16% F1, with a higher recall than the TF-IDF baseline but lower precision. The results indicate that contextual pretrained representations provide a substantial benefit for this task in our experimental setting.

Keywords: e-mail intent, actionable e-mail detection, text classification, TF-IDF, logistic regression, Word2Vec, BiLSTM, BERT

Table of Contents

1. Introduction.....	3
2. Problem Statement.....	3
3. Objectives	4
4. Dataset.....	4
4.1 Source and Public Description.....	4
4.2 Dataset as Used in This Project	5
5. Methodology	5
5.1 Experimental Design.....	5
5.2 Implementation and Environment.....	6
6. Data Preprocessing.....	6
6.1 Text Cleaning.....	6
6.2 Removal of Invalid and Duplicate Examples	7
6.3 Data Splitting	7
6.4 Model-Specific Tokenisation.....	8
7. Model Architectures.....	8
7.1 Model A: TF-IDF + Logistic Regression	8
7.2 Model B: Pretrained Word2Vec + BiLSTM.....	9
7.3 Model C: Pretrained BERT (bert-base-uncased)	10
8. Training.....	10
8.1 Model A	10
8.2 Models B and C	10
8.3 Training Behaviour	11
9. Evaluation Metrics	12
10. Results and Comparison	13
10.1 Test-Set Results	13
10.2 Confusion Matrices and Error Types	13
10.3 Validation versus Test Performance	14
11. Discussion	14
12. Limitations	15
13. Conclusion	16
References.....	17

1. Introduction

E-mail remains a primary channel for professional and personal communication. A substantial fraction of the text a user receives, however, does not require any response: newsletters, notifications, promotional content and purely informative statements are mixed with requests, proposals and questions that do require the recipient's action. Identifying the latter quickly is important, because an overlooked request can result in a missed deadline, an unanswered question or a delayed decision.

Natural Language Processing (NLP) offers a way to support this triage automatically. The task can be framed as a supervised binary classification problem in which a model reads a piece of e-mail text and predicts whether it is actionable. This framing is closely related to prior work on e-mail speech acts, in which sentences or messages are classified into communicative categories such as *request*, *propose* and *commit* [2].

This report presents **MailSense**, a project that implements and compares three representative approaches to this problem: a classical sparse-feature model (TF-IDF + Logistic Regression), a recurrent neural model with static pretrained word embeddings (Word2Vec + BiLSTM), and a fine-tuned pretrained Transformer (BERT). The emphasis is on a controlled comparison: all models are trained and evaluated on identical data under identical evaluation procedures, so that differences in the reported results can be attributed to the modelling approach rather than to differences in data handling.

Throughout the report, a clear distinction is maintained between (i) the description of the external dataset, which is taken from its public source [1], and (ii) the experimental work carried out in this project, which is documented by the project's source code, configuration files and saved result files.

2. Problem Statement

Given a piece of e-mail text x , the goal is to learn a function $f(x) \in \{\text{Yes}, \text{No}\}$ that predicts whether the text requires the recipient's action or attention:

- **Yes (Actionable):** the text contains content that asks for or proposes an action by the recipient (for example, a request or a proposal).
- **No (Non-Actionable):** the text does not require the recipient to act.

The positive class is **Actionable (Yes)**. Consequently, a *false negative* is an actionable text that the model predicts as non-actionable. For the intended application this is the more costly error, since it could cause the user to miss content that needs attention; recall on the actionable class and the number of false negatives are therefore reported explicitly alongside the other metrics.

Because the dataset consists of individual e-mail sentences or short snippets rather than complete structured e-mails with separate subject and body fields, the task addressed in this

project is more precisely **actionable e-mail-content classification** at the sentence/snippet level.

The research question guiding the project is: *How do a traditional TF-IDF-based classifier, an LSTM-based neural model with pretrained Word2Vec embeddings, and a pretrained Transformer-based BERT model compare for actionable e-mail-content classification on the same dataset, split and evaluation protocol?*

3. Objectives

The objectives of the project are:

1. To prepare a clean, de-duplicated version of the Parakweet Labs Email Intent training file and a single stratified train/validation/test split shared by all models.
2. To implement a classical baseline using TF-IDF features and Logistic Regression.
3. To implement a neural sequence model using a bidirectional LSTM initialised with pretrained Word2Vec embeddings.
4. To fine-tune a pretrained Transformer model (bert-base-uncased) for the same binary task.
5. To evaluate all three models on the same held-out test set using accuracy, precision, recall, F1-score, ROC-AUC and confusion-matrix counts, with Actionable (Yes) as the positive class.
6. To compare and discuss the results, including the types of errors each model makes, and to identify the limitations of the study.

4. Dataset

4.1 Source and Public Description

This subsection summarises the dataset as described by its publishers; it does not describe work performed in this project.

The data used is the **Parakweet Labs Email Intent Dataset** [1], released on GitHub by Parakweet Labs, Inc. in 2014 under the Apache License 2.0. The repository describes it as "labeled training and test data for email intent machine learning (for sentence-level speech acts)". According to the dataset wiki, the data comes from the Enron e-mail corpus [3], and each labelled example is one sentence from an e-mail. The wiki defines "intent" as corresponding primarily to the speech-act categories *request* and *propose* of Cohen, Carvalho and Mitchell [2], and states that some *commit* sentences containing date/time information were also labelled positive. The wiki does not document the annotation procedure or inter-annotator agreement.

The repository provides a training file (Ask0729-fixed.txt) and a separate test file (testSet-qualifiedBatch-fixed.txt). **This project uses only the training file, Ask0729-fixed.txt**, which was obtained from the following location:

<https://raw.githubusercontent.com/ParakweetLabs/EmailIntentDataSet/master/src/resources/Ask0729-fixed.txt>

Each line of the file contains a label (Yes or No), a tab character and the text of one example.

4.2 Dataset as Used in This Project

The copy of Ask0729-fixed.txt stored in the project repository (data/Ask0729-fixed.txt) is byte-for-byte identical to the file at the URL above. Parsing it with the project's data-preparation script yields the counts in Table 1; all lines had a valid label and format, so no rows were rejected at the parsing stage.

Table 1. Class distribution of Ask0729-fixed.txt as parsed in this project

Label	Meaning	Examples	Share
Yes	Actionable	1,719	47.0%
No	Non-Actionable	1,938	53.0%
Total		3,657	100.0%

Source: results/data_quality_report.json (raw_rows, raw_class_distribution).

The dataset wiki reports 4,213 training cases with 1,631 positives. These figures differ from the 3,657 examples (1,719 Yes) contained in the file as currently published at the URL above; the reason for the difference is not documented, and all figures in this report refer to the file as actually used.

The classes are close to balanced (47.0% actionable). Manual inspection of the file shows that, in addition to ordinary correspondence, it contains promotional and notification text (for example, sentences asking the reader to add a sender to their address book). After cleaning (Section 6), the examples have a mean length of 16.6 whitespace-separated words (median 14, maximum 222); actionable examples are slightly shorter on average (15.4 words) than non-actionable ones (17.6 words).

5. Methodology

5.1 Experimental Design

The study is a controlled comparison of three text-classification pipelines. To ensure that differences in results reflect the models and not the data handling, the following conditions are shared by all three models:

- the same cleaned dataset and the same original Yes/No labels, mapped deterministically to 1/0;
- the same stratified train/validation/test split, stored once as CSV files and read by every model;
- the same random seed (42) for Python, NumPy and PyTorch, with deterministic cuDNN settings;
- the same evaluation module, the same positive class (Actionable = Yes) and the same decision threshold of 0.5 on the predicted probability of the actionable class;
- the validation set is used for model selection (early stopping and checkpoint selection for the neural models), and the test set is used only once, for final evaluation after training.

The three pipelines are summarised in Table 2.

Table 2. Overview of the three compared pipelines

Model	Text representation	Classifier
A: TF-IDF + LR	Sparse TF-IDF vector of word unigrams and bigrams	Logistic Regression
B: Word2Vec + BiLSTM	Sequence of 300-d pretrained Word2Vec vectors (fine-tuned)	Bidirectional LSTM + linear layer
C: BERT	Contextual WordPiece representations from bert-base-uncased	Linear classification head; full fine-tuning

5.2 Implementation and Environment

The project is implemented in Python. Model A uses scikit-learn [9] (version 1.6.1); Model B uses PyTorch [10] and gensim [12]; Model C uses PyTorch and the Hugging Face Transformers library [11]. Hyperparameters for each model are stored in JSON configuration files (configs/tfidf.json, configs/lstm.json, configs/bert.json). The final experiments were executed in a Kaggle notebook (notebooks/training/mailesense-final-training.ipynb) with PyTorch 2.10.0 on an NVIDIA Tesla T4 GPU. The notebook runs data preparation and then trains and evaluates the three models in sequence; all metrics reported in Section 10 are taken from the result files produced by that run (results/*/metrics.json and results/final_test_comparison.csv).

6. Data Preprocessing

6.1 Text Cleaning

A shared cleaning function (src/preprocessing/clean.py) is applied to every example before splitting. The steps, applied in order, are:

1. Unicode normalisation (NFKC form).
2. Decoding of HTML character entities.

3. Replacement of URLs (including angle-bracketed URLs) with the placeholder token \$LINK.
4. Replacement of e-mail addresses with the placeholder token \$EMAIL.
5. Removal of any remaining HTML-like tags.
6. Replacement of numbers (including numbers containing commas, dots or hyphens) with the placeholder token \$NUM.
7. Collapsing of repeated whitespace and trimming.

No stop-word removal, stemming or lemmatisation is applied. Function words and punctuation are retained deliberately, since words such as *please*, *could* and *you* and the question mark can carry information about whether a sentence is a request. The placeholder tokens reduce sparsity from unique URLs, addresses and numbers while preserving the fact that such an item was present.

6.2 Removal of Invalid and Duplicate Examples

After cleaning, the preparation script (src/preprocessing/prepare_data.py) applies three filters: (i) examples containing fewer than two alphabetic characters are removed as empty or meaningless; (ii) cleaned texts that occur with *conflicting* labels are removed entirely, since the correct label cannot be determined; and (iii) exact duplicates of a cleaned text with the same label are removed, keeping the first occurrence. Every removed example is logged with its reason in results/removed_examples.csv.

In the final run, filters (i) and (ii) removed no examples. Filter (iii) removed **6 duplicate examples** (5 No and 1 Yes). Several of these became duplicates only after e-mail addresses were replaced by \$EMAIL. The final dataset therefore contains **3,651 examples: 1,718 Actionable and 1,933 Non-Actionable**.

6.3 Data Splitting

The cleaned data is divided into training (70%), validation (15%) and test (15%) sets by two successive stratified random splits (scikit-learn train_test_split, random_state = 42), which preserves the class proportions in each subset. The resulting sizes are given in Table 3. The split is saved once to data/splits/ and reused unchanged by all three models.

Table 3. Stratified data split used by all models

Split	Actionable (Yes)	Non-Actionable (No)	Total	Share
Training	1,202	1,353	2,555	70.0%
Validation	258	290	548	15.0%
Test	258	290	548	15.0%
Total	1,718	1,933	3,651	100.0%

Source: results/data_quality_report.json.

6.4 Model-Specific Tokenisation

After the shared cleaning step, each model applies its own tokenisation:

- **Model A** uses the scikit-learn TfidfVectorizer tokeniser with lower-casing and Unicode accent stripping. Its default token pattern keeps alphanumeric tokens of two or more characters, so punctuation is not used as a feature and the placeholders appear as the lower-cased words *link*, *email* and *num*.
- **Model B** uses a regular-expression tokeniser (`src/lstm/vocab.py`) that lower-cases words, keeps the \$LINK, \$EMAIL and \$NUM placeholders intact, and keeps the punctuation marks ? ! . , ; : as separate tokens. The vocabulary is built from the training split only (minimum frequency 2), giving 2,450 entries including the padding and unknown tokens; validation and test words not in this vocabulary are mapped to the unknown token. Sequences are truncated or padded to 64 tokens.
- **Model C** uses the WordPiece tokeniser of bert-base-uncased with truncation and padding to a maximum length of 64 tokens.

Because the maximum sequence length is 64 tokens for Models B and C, the small number of unusually long examples are truncated; 13 of the 3,651 cleaned examples contain more than 64 whitespace-separated words.

7. Model Architectures

7.1 Model A: TF-IDF + Logistic Regression

TF-IDF (term frequency–inverse document frequency) [4] represents each text as a sparse vector in which each dimension corresponds to a word n-gram, weighted by how often it appears in the text and down-weighted by how common it is across the training corpus. The vectoriser is fitted on the training split only. Unigrams and bigrams are used, so that short phrases such as *please send* or *let me* can serve as features. Sub-linear term-frequency scaling ($1 + \log \text{tf}$) is applied, n-grams occurring in fewer than 2 training documents or in more than 90% of them are discarded, and the resulting vocabulary contains **6,572 features**.

A Logistic Regression classifier is trained on these vectors. It models the probability of the actionable class as the logistic (sigmoid) function of a weighted sum of the features, with L2 regularisation (inverse strength $C = 1.0$). Class weights are set to *balanced*, which scales each class's contribution to the loss inversely to its frequency. The liblinear solver [15] is used with a maximum of 2,000 iterations. The configuration is summarised in Table 4.

Table 4. Configuration of Model A

Component	Setting
N-gram range	(1, 2) — unigrams and bigrams
min_df / max_df	2 / 0.9
Term-frequency scaling	Sub-linear ($1 + \log \text{tf}$)
Lower-casing / stop words	Yes / none removed
Vocabulary size (fitted on training split)	6,572
Classifier	Logistic Regression, L2 penalty, $C = 1.0$, liblinear solver
Class weight	Balanced
Maximum iterations	2,000

7.2 Model B: Pretrained Word2Vec + BiLSTM

Model B is a recurrent neural network that reads a text as a sequence of word vectors. Its components are:

- **Embedding layer.** Each token ID is mapped to a 300-dimensional vector. The embedding matrix is initialised from the pretrained **Google News Word2Vec** vectors [5] (3 million words, 300 dimensions). To keep the project self-contained, the vectors for the words in the training vocabulary were extracted from the full model into a small file (embeddings/word2vec.bin). Of the 2,448 regular vocabulary words, **2,338 (95.51%)** were found in Word2Vec; the remaining words (including the placeholder tokens) are initialised randomly from $N(0, 0.05^2)$, and the padding and unknown vectors are initialised to zero. The embeddings are **trainable**, i.e. fine-tuned during training.
- **Bidirectional LSTM.** A single-layer bidirectional LSTM [6, 7] with a hidden size of 128 per direction processes the sequence. Padded positions are excluded using packed sequences, so each direction stops at the true end of the text.
- **Classifier.** The final forward and backward hidden states are concatenated into a 256-dimensional vector, passed through dropout ($p = 0.3$) and a linear layer that outputs two logits (No, Yes). Dropout with $p = 0.3$ is also applied to the embeddings. A softmax over the two logits gives the probability of the actionable class.

The model has **1,175,834 trainable parameters**, of which the majority belong to the embedding layer (Table 5).

Table 5. Parameter count of Model B

Layer	Shape / description	Parameters
Embedding	$2,450 \times 300$	735,000
BiLSTM	input 300, hidden 128, two directions	440,320
Linear output layer	$256 \rightarrow 2$	514
Total		1,175,834

Total as recorded in results/lstm/metrics.json; per-layer counts follow from the layer dimensions.

7.3 Model C: Pretrained BERT (bert-base-uncased)

BERT (Bidirectional Encoder Representations from Transformers) [8] is a Transformer encoder pretrained on large unlabelled English corpora with masked-language-modelling and next-sentence-prediction objectives. Unlike static Word2Vec vectors, BERT produces a *contextual* representation of each token that depends on the whole input sentence. The bert-base-uncased model has 12 Transformer layers, a hidden size of 768 and 12 self-attention heads, and operates on lower-cased WordPiece tokens.

The model is loaded with the Hugging Face AutoModelForSequenceClassification class with two output labels. This adds the standard sequence-classification head, a dropout layer followed by a linear layer on the pooled representation of the [CLS] token. No layers are frozen and the architecture is not modified: all **109,483,778 parameters** are fine-tuned on the task.

8. Training

8.1 Model A

Model A is trained in a single step: the TF-IDF vectoriser and the Logistic Regression classifier are fitted as one scikit-learn pipeline on the 2,555 training examples. The validation set is used only to report validation metrics; no hyperparameter search is carried out in the final run, which uses the fixed configuration in Table 4.

8.2 Models B and C

Both neural models are trained with mini-batch gradient descent to minimise the cross-entropy loss over the two classes. After every epoch, the model is evaluated on the validation set; the checkpoint with the **highest validation F1-score** (actionable class) is saved, and training stops early if the validation F1 does not improve for a given number of consecutive epochs (the *patience*). After training, the best checkpoint is reloaded and evaluated on the validation and test sets. The training hyperparameters are listed in Table 6.

Table 6. Training hyperparameters of the neural models

Hyperparameter	Model B: Word2Vec + BiLSTM	Model C: BERT
Optimiser	Adam [14]	AdamW [13]
Learning rate	1×10^{-3} (constant)	2×10^{-5} , linear decay with 10% warm-up
Weight decay	0	0.01
Batch size	32	16
Maximum epochs	20	4
Early-stopping patience	4 epochs	2 epochs
Gradient-norm clipping	5.0	1.0
Maximum sequence length	64 tokens	64 WordPiece tokens
Dropout	0.3	Model default
Model-selection criterion	Validation F1 (Yes)	Validation F1 (Yes)

8.3 Training Behaviour

Tables 7 and 8 reproduce the per-epoch training history recorded during the final run.

Table 7. Training history of Model B (Word2Vec + BiLSTM)

Epoch	Training loss	Validation loss	Validation accuracy	Validation F1
1	0.5618	0.4428	81.20%	78.41%
2	0.3761	0.4507	81.75%	79.25%
3 (best)	0.2783	0.4073	82.48%	81.75%
4	0.1981	0.4670	82.48%	80.41%
5	0.1455	0.5336	81.39%	80.16%
6	0.1023	0.5662	80.84%	80.15%
7	0.0782	0.6504	82.48%	81.47%

Source: results/lstm/training_history.csv.

For Model B, the best validation F1 (81.75%) was reached at epoch 3. The validation F1 did not improve over the following four epochs, so training stopped early at epoch 7 and the epoch-3 checkpoint was used. Over these epochs the training loss continued to fall (from 0.278 to 0.078) while the validation loss rose (from 0.407 to 0.650), indicating that the model began to overfit the training data after epoch 3. Each epoch took approximately one second on the T4 GPU.

Table 8. Training history of Model C (BERT)

Epoch	Training loss	Validation loss	Validation accuracy	Validation F1
1	0.5173	0.3550	83.21%	83.69%
2	0.2714	0.3678	85.77%	83.95%
3	0.1532	0.4580	86.68%	86.04%
4 (best)	0.0775	0.5395	87.77%	87.38%

Source: results/bert/training_history.csv.

For Model C, validation accuracy and F1 improved in every epoch, so early stopping was not triggered and the checkpoint from the final (fourth) epoch was used, with a validation F1 of 87.38%. The validation loss increased after the first epoch even as validation accuracy and F1 improved, which suggests that the model became more confident on the examples it misclassified; checkpoint selection was based on F1 rather than loss. Each epoch took approximately 30–34 seconds on the T4 GPU.

9. Evaluation Metrics

All models are evaluated by the same module (src/evaluation/metrics.py) on the same 548-example test set (258 Actionable, 290 Non-Actionable). A prediction is *Actionable* when the model's predicted probability of the actionable class is at least 0.5. With Actionable (Yes) as the positive class, the four confusion-matrix counts are:

- **True Positive (TP):** actionable text predicted as actionable.
- **True Negative (TN):** non-actionable text predicted as non-actionable.
- **False Positive (FP):** non-actionable text predicted as actionable (an unnecessary alert).
- **False Negative (FN):** actionable text predicted as non-actionable (a missed item).

From these counts the following metrics are computed:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

$$Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

$$F1 = 2 \times Precision \times Recall / (Precision + Recall)$$

Accuracy measures the overall proportion of correct predictions. Precision measures how many of the texts flagged as actionable are truly actionable, while recall measures how many of the truly actionable texts are found. The F1-score is the harmonic mean of precision and recall and is the primary single-number metric for the actionable class; it is also the criterion used for model selection on the validation set. **ROC-AUC** is the area under the receiver operating characteristic curve, computed from the predicted probabilities; it measures how well a model ranks actionable texts above non-actionable ones across all possible thresholds, independently

of the 0.5 threshold. Precision, recall and F1 refer to the actionable class unless stated otherwise; macro-averaged F1 over both classes is also recorded.

10. Results and Comparison

10.1 Test-Set Results

Table 9 reports the final results of the three models on the held-out test set. These are the results of the single final run described in Sections 5–8.

Table 9. Test-set results (n = 548; positive class = Actionable)

Model	Accuracy	Precision	Recall	F1	ROC-AUC
A: TF-IDF + Logistic Regression	77.74%	76.98%	75.19%	76.08%	86.07%
B: Word2Vec + BiLSTM	74.82%	71.74%	76.74%	74.16%	86.54%
C: BERT (bert-base-uncased)	84.85%	82.05%	86.82%	84.37%	93.16%

Best value in each column shown in bold. Source: results/final_test_comparison.csv.

BERT achieved the highest value on every metric. Compared with the TF-IDF baseline, it improved accuracy by 7.11 percentage points, F1 by 8.29 points, recall by 11.63 points and ROC-AUC by 7.09 points. The TF-IDF + Logistic Regression baseline outperformed the Word2Vec + BiLSTM model on accuracy (by 2.92 points), precision (by 5.24 points) and F1 (by 1.92 points), whereas the BiLSTM obtained a higher recall (by 1.55 points) and a slightly higher ROC-AUC (by 0.47 points).

10.2 Confusion Matrices and Error Types

Table 10 gives the confusion-matrix counts on the test set.

Table 10. Test-set confusion-matrix counts

Model	TP	TN	FP	FN	Errors (FP + FN)
A: TF-IDF + Logistic Regression	194	232	58	64	122
B: Word2Vec + BiLSTM	198	212	78	60	138
C: BERT (bert-base-uncased)	224	241	49	34	83

Source: results/*/confusion_matrix_test.csv. Rows of each confusion matrix sum to 258 actionable and 290 non-actionable examples.

For the application, the false-negative count is the most important error measure. BERT missed 34 of the 258 actionable test examples, compared with 64 for TF-IDF + Logistic Regression and 60 for the BiLSTM; this is 30 fewer missed actionable examples than the baseline (a reduction of 47%). BERT also produced the fewest false positives (49). The BiLSTM produced the most false positives (78), which explains its lower precision despite its slightly better recall than the baseline.

Table 11 reports the metrics for the Non-Actionable class and the macro-averaged F1, which weights both classes equally.

Table 11. Test-set metrics for the Non-Actionable class and macro-averaged F1

Model	Precision (No)	Recall (No)	F1 (No)	Macro F1
A: TF-IDF + Logistic Regression	78.38%	80.00%	79.18%	77.63%
B: Word2Vec + BiLSTM	77.94%	73.10%	75.44%	74.80%
C: BERT (bert-base-uncased)	87.64%	83.10%	85.31%	84.84%

Non-Actionable metrics computed from the counts in Table 10; macro F1 from results/*/test_summary.csv.

10.3 Validation versus Test Performance

Table 12 compares the F1-score of each model on the validation set (used for model selection) and on the test set.

Table 12. F1-score (Actionable) on the validation and test sets

Model	Validation F1	Test F1	Difference (points)
A: TF-IDF + Logistic Regression	78.02%	76.08%	−1.94
B: Word2Vec + BiLSTM	81.75%	74.16%	−7.59
C: BERT (bert-base-uncased)	87.38%	84.37%	−3.01

Source: results/*/metrics.json.

All models scored lower on the test set than on the validation set, which is expected for the neural models because their checkpoints were selected to maximise validation F1. The drop is largest for the BiLSTM (7.59 points): on the validation set it outperformed the TF-IDF baseline (81.75% vs. 78.02% F1), but on the test set the ordering was reversed. BERT remained the best model on both sets.

11. Discussion

Effect of contextual pretraining. The clearest result of the study is the advantage of the fine-tuned BERT model, which was best on all reported metrics on both the validation and the test set. Whether a sentence is actionable often depends on its grammatical form and on the interaction between words rather than on individual words alone; for example, the same verb can occur in a request ("Could you send the file?") and in a report ("I sent the file."). BERT represents each token in the context of the whole sentence and is pretrained on large amounts of English text, which plausibly explains its better handling of such cases compared with bag-of-n-grams features and static word vectors. The gain came at a considerably higher computational cost: roughly 109.5 million fine-tuned parameters and about 30 seconds per training epoch on a T4 GPU, compared with about one second per epoch for the BiLSTM.

Strength of the classical baseline. TF-IDF + Logistic Regression proved a competitive baseline, outperforming the BiLSTM on test accuracy, precision and F1. Actionable sentences frequently contain explicit lexical cues (for example, *please*, *let me know*, *can you*), which unigram and bigram features capture directly. On a training set of only 2,555 examples, a linear model with regularisation and relatively few effective parameters is less prone to overfitting than a neural network with over one million parameters.

Behaviour of the BiLSTM. The BiLSTM benefited from pretrained Word2Vec initialisation (95.5% vocabulary coverage) and reached the second-best validation F1, but it generalised less well to the test set. The training history (Table 7) shows the validation loss increasing while the training loss continued to decrease after epoch 3, and the gap between validation and test F1 (7.59 points) is the largest of the three models. Together these observations suggest that the BiLSTM's validation score was optimistic and that its selected checkpoint was sensitive to the particular validation sample. Its test errors were skewed towards false positives (78), giving it higher recall but lower precision than the baseline.

Ranking versus thresholded performance. The BiLSTM had a slightly higher ROC-AUC than the TF-IDF baseline (86.54% vs. 86.07%) while having lower accuracy and F1. ROC-AUC evaluates the ranking of examples across all thresholds, whereas accuracy, precision, recall and F1 depend on the fixed threshold of 0.5. This indicates that the two models separate the classes comparably well overall, and that their differences at the 0.5 threshold partly reflect where each model's probabilities fall relative to that threshold. Threshold tuning was not performed in this project.

Implications for the application. Since missing an actionable e-mail is the more costly error, recall and the false-negative count are particularly relevant. BERT offers the best trade-off in this respect, finding 86.8% of actionable test examples while also having the highest precision. Where computational resources are constrained, the TF-IDF baseline offers a reasonable lower-cost alternative, at the cost of roughly twice as many missed actionable examples (64 vs. 34) in our test set.

12. Limitations

The conclusions of this study should be read in light of the following limitations:

- **Single run.** Each model was trained once with a single random seed (42) and a single train/validation/test split. The variability of the results across seeds and splits was not measured, and no statistical significance tests were performed. With a test set of 548 examples, small differences between models, such as the 0.47-point ROC-AUC difference between Models A and B, should not be treated as conclusive.
- **Fixed hyperparameters.** Each model was evaluated with one fixed configuration (Tables 4 and 6). No systematic hyperparameter search is documented in the project files, so each model may not be at its best achievable performance.

- **Sentence-level data.** The examples are individual sentences or short snippets, not complete e-mails. The models therefore do not use the context of the surrounding message, the subject line, sender information or the conversation thread, all of which could affect whether an e-mail as a whole requires action.
- **Dataset scope and labels.** Only the training file of the Parakweet Labs dataset (Ask0729-fixed.txt) was used; the dataset's separate official test file was not evaluated. The annotation procedure and inter-annotator agreement are not documented by the dataset publishers, and the number of examples in the file differs from the figure stated in the dataset wiki. The label definition (primarily *request* and *propose* speech acts) is a specific operationalisation of "actionable". The data is English-only, and the file includes promotional and notification text in addition to ordinary correspondence.
- **Fixed decision threshold and truncation.** All models used a threshold of 0.5; a threshold chosen to favour recall might reduce false negatives at the cost of more false positives. Inputs to Models B and C were truncated to 64 tokens, which affects a small number of long examples.
- **Limited error analysis.** The analysis in this report is based on aggregate metrics and confusion-matrix counts. A systematic qualitative analysis of the misclassified examples, although supported by the saved per-example test predictions, was not carried out.

13. Conclusion

This project developed MailSense, a binary classifier that identifies actionable e-mail content, and compared three NLP approaches under a controlled experimental protocol on the Parakweet Labs Email Intent Dataset (Ask0729-fixed.txt). After cleaning and de-duplication, 3,651 examples were split into stratified training, validation and test sets that were shared by all models.

On the held-out test set, the fine-tuned BERT model (bert-base-uncased) achieved the best performance on every metric, with 84.85% accuracy, 84.37% F1 and 93.16% ROC-AUC, and missed the fewest actionable examples (34 of 258). The TF-IDF + Logistic Regression baseline achieved 77.74% accuracy and 76.08% F1, and the Word2Vec + BiLSTM model achieved 74.82% accuracy and 74.16% F1, with higher recall but lower precision than the baseline and the largest drop from validation to test performance.

These findings indicate that, for sentence-level actionable e-mail detection on this dataset, contextual representations from a pretrained Transformer provide a substantial improvement over both sparse lexical features and static word embeddings, while a simple TF-IDF model remains a strong, low-cost baseline. Natural extensions of this work, motivated by the limitations above, include repeating the experiments over multiple seeds with significance testing, evaluating on the dataset's official test file, tuning the decision threshold for higher recall, and extending the task from single sentences to complete e-mail messages.

References

- [1] Parakweet Labs, Inc., "EmailIntentDataSet: Labeled training and test data for email intent machine learning (for sentence-level speech acts)," GitHub repository, 2014. Apache License 2.0. Available: <https://github.com/ParakweetLabs/EmailIntentDataSet>. Wiki: <https://github.com/ParakweetLabs/EmailIntentDataSet/wiki>. Training file used in this project: <https://raw.githubusercontent.com/ParakweetLabs/EmailIntentDataSet/master/src/resources/Ask0729-fixed.txt>.
- [2] W. W. Cohen, V. R. Carvalho, and T. M. Mitchell, "Learning to classify email into 'speech acts'," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2004, pp. 309–316.
- [3] B. Klimt and Y. Yang, "The Enron corpus: A new dataset for email classification research," in *Proc. European Conf. Machine Learning (ECML)*, 2004, pp. 217–226.
- [4] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [5] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 26, 2013.
- [6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [7] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.
- [8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [9] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [10] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [11] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP 2020: System Demonstrations*, 2020, pp. 38–45.
- [12] R. Řehůřek and P. Sojka, "Software framework for topic modelling with large corpora," in *Proc. LREC 2010 Workshop on New Challenges for NLP Frameworks*, 2010, pp. 45–50.

- [13] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.
- [14] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.
- [15] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, and C.-J. Lin, "LIBLINEAR: A library for large linear classification," *Journal of Machine Learning Research*, vol. 9, pp. 1871–1874, 2008.